

Object Stereotypes for Spring Boot Applications

Domain Objects · Validators · Service Layers · Data Access Classes

NOTE: This is for quick awareness of some "types" of objects that may exist in your solution. It is NOT a cookbook and again, is here for awareness, not reference

A practical guide to writing code that fits the architecture



What we'll cover

1

Why Stereotypes Matter

The cost of tangled code, and what structure buys you

2

The Layered Architecture

How a request flows from controller to database

3

Domain Objects

Modeling the business, not the database row

4

Validators

Keeping bad data out, consistently

5

Service Layer

Where business logic and transactions live

6

Data Access Classes

Repositories, and why they stay dumb

7

Anti-Patterns & Conventions

Mistakes to avoid, and how we name things

MOTIVATION

Why object stereotypes matter

In a small script, it doesn't matter where a line of code lives. In an enterprise codebase touched by dozens of developers over years, it matters enormously.

A stereotype is a well-known role an object plays: domain object, validator, service, or DAO. When every class fits a recognized role, new developers can predict where logic lives before they've read a single line.



Predictability

Anyone can guess where logic lives



Testability

Each layer is tested in isolation



Safe change

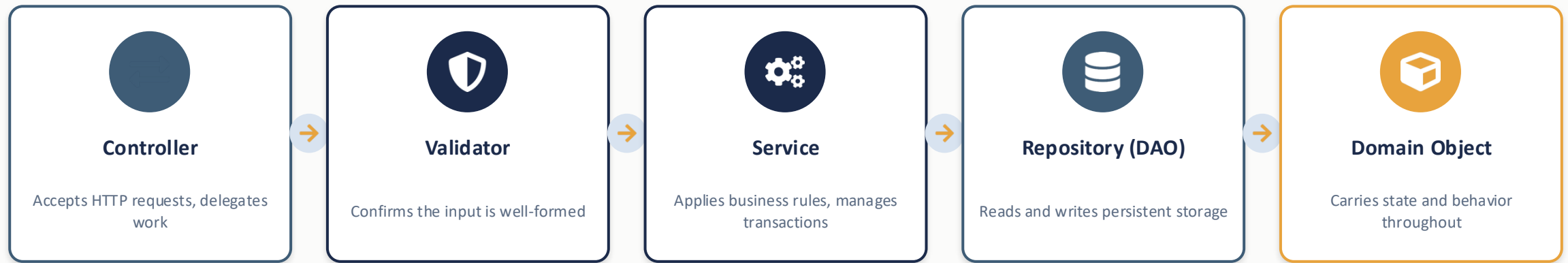
Swap a database without touching business rules



Without structure...

- SQL queries scattered inside web controllers
- Validation logic duplicated in five places
- Business rules mixed with HTTP request handling
- A single 2,000-line "God class" nobody wants to touch
- Tests that require a live database for everything

How a request flows through the layers



The domain object is the one thing every layer touches — it moves from repository, through service, to controller, carrying the same shape the whole way.

The golden rule: each layer only calls the layer directly below it. A controller never touches a repository, and a repository never calls a service.

STEREOTYPE 1

Domain Objects



A domain object represents a real concept in the business — an Order, a Customer, an Invoice. It holds state and the behavior that belongs to that state.

Named after the business, not the table

"Customer", not "CustomerRow" or "CustomerEntity"

Owns its invariants

An Order should not let itself hold a negative total

Free of framework and persistence concerns

No HTTP, no SQL, no JSON annotations bleeding through

Small and focused

One concept per class — resist the urge to grow a God object

Common confusion

A JPA @Entity is a persistence mapping, not automatically a well-formed domain object. It's fine for many apps to let the entity BE the domain object — but keep it disciplined: no getters that leak mutable collections, no setters that skip validation, and no controller code reaching into it directly.

Rule of thumb: if a class needs both @Entity and business rule methods, that's fine — just don't let it also parse JSON or format HTTP responses.

Domain Object — Order

```
@Entity
@Table(name = "orders")
public class Order {

    @Id @GeneratedValue
    private Long id;

    private CustomerId customerId;
    private List<OrderLine> lines = new ArrayList<>();
    private OrderStatus status = OrderStatus.DRAFT;

    // behavior lives with the state it protects
    public void addLine(OrderLine line) {
        if (status != OrderStatus.DRAFT) {
            throw new IllegalStateException("Cannot modify a submitted order");
        }
        lines.add(line);
    }

    public BigDecimal total() {
        return lines.stream().map(OrderLine::subtotal)
            .reduce(BigDecimal.ZERO, BigDecimal::add);
    }
}
```

Notice: the class enforces its own rule (no edits after submission) instead of trusting every caller to remember.

Domain objects are not DTOs

A DTO (Data Transfer Object) shapes data for the wire — a request or response body. A domain object shapes data for the business rules. Conflating the two is one of the most common early mistakes.



Domain Object

- Lives in the service & repository layers
- Enforces business invariants
- Shaped by the business concept
- Long-lived, reused across use cases



DTO

- Lives at the controller boundary
- No business logic — just fields
- Shaped by the API contract
- One per request/response shape

Validators



A validator answers exactly one question: is this input acceptable? It never mutates state, never calls the database, and never decides what happens next — it only reports problems.

Declarative

Bean Validation annotations (@NotNull, @Size, @Email) on a DTO field, checked automatically by Spring at the controller boundary.

Programmatic

A dedicated *Validator class for rules too complex for annotations — cross-field checks, or rules that depend on other data.

Why keep validators separate from services?

Because the same rule set often needs to run in more than one place — a REST endpoint and a batch import job, for instance — and because a validator with no side effects is trivial to unit test with plain objects, no Spring context required.

Validator — OrderValidator

```
@Component
public class OrderValidator {

    public ValidationResult validate(CreateOrderRequest req) {
        List<String> errors = new ArrayList<>();

        if (req.getLines() == null || req.getLines().isEmpty()) {
            errors.add("Order must contain at least one line item");
        }
        if (req.getCustomerId() == null) {
            errors.add("customerId is required");
        }
        req.getLines().forEach(line -> {
            if (line.getQuantity() <= 0) {
                errors.add("Quantity must be positive: " + line.getSku());
            }
        });

        return ValidationResult.of(errors);
    }
}
```

No database calls, no exceptions for control flow — just data in, a list of problems out. That makes it fast and easy to test.

STEREOTYPE 3

Service Layer



The service layer is where business use cases live: "place an order," "close an account," "apply a discount." It coordinates domain objects and repositories, and owns the transaction boundary.

One method per use case

`placeOrder()`, not a generic `update()` that does five different things

Owns @Transactional

A single service method defines where a unit of work begins and ends

Talks to repositories, not to SQL

Persistence detail is delegated, never inlined

Stateless

A service instance holds no per-request state between calls

Service — OrderService

```
@Service
public class OrderService {

    private final OrderRepository orders;
    private final InventoryClient inventory;

    // constructor injection – fields stay final
    public OrderService(OrderRepository orders, InventoryClient inventory) {
        this.orders = orders;
        this.inventory = inventory;
    }

    @Transactional
    public Order placeOrder(CustomerId customer, List<OrderLine> lines) {
        Order order = new Order(customer);
        lines.forEach(order::addLine);

        inventory.reserve(order);
        return orders.save(order);
    }
}
```

The reservation call and the save happen in one transaction — if either fails, both roll back.

Data Access Classes



A repository (or DAO) is solely responsible for moving domain objects in and out of storage. It knows how to query — it does not know why the caller is asking, or what to do with the answer.

Spring Data JPA

For typical CRUD and simple queries, extend `JpaRepository` and let Spring generate the implementation from method names — no boilerplate SQL to maintain.

Use `@Query` (JPQL or native SQL) when the derived-method-name syntax gets unreadable.

The one rule that matters

A repository never contains business logic. No discount math, no status transitions, no "if premium customer then..." — that all belongs one layer up, in the service.

If you find yourself writing "and()" logic branches inside a repository method, that's a signal the logic wants to live in the service instead.

Repository — OrderRepository

```
@Repository
public interface OrderRepository extends JpaRepository<Order, Long> {

    // Spring Data derives the query from the method name
    List<Order> findByCustomerIdAndStatus(
        CustomerId customerId, OrderStatus status);

    // drop to JPQL when the method name would get unreadable
    @Query("select o from Order o where o.total > :minTotal")
    List<Order> findHighValueOrders(@Param("minTotal") BigDecimal minTotal);
}
```

That's it — an interface, no implementation to write. Spring Data JPA generates it at startup.

Putting it all together: placing an order

1

Controller

receives POST /orders, deserializes the JSON into a CreateOrderRequest DTO

2

Validator

checks the DTO for missing fields and bad quantities before anything else runs

3

Service

builds an Order domain object, applies business rules, opens the transaction

4

Repository

persists the Order and returns the saved entity with its generated id

5

Controller

maps the saved Order back to a response DTO and returns 201 Created

Common anti-patterns



Fat controller

Business logic and SQL written directly inside a `@RestController` method



Repository with business branches

if/else business decisions embedded in a query method



Service returning entities straight to the wire

Skipping DTOs leaks persistence details (and lazy-loading errors) into the API



Anemic everything, service does it all

Domain objects are bare getters/setters; every rule lives in one giant service class



Validation duplicated per endpoint

The same field checks copy-pasted instead of shared in one validator



Transactions started in the wrong layer

`@Transactional` on a controller or repository method instead of the service

Naming and package structure

```
com.acme.orders
├── domain/
│   ├── Order.java
│   ├── OrderLine.java
│   └── OrderStatus.java
├── validation/
│   └── OrderValidator.java
├── service/
│   └── OrderService.java
├── repository/
│   └── OrderRepository.java
└── web/
    ├── OrderController.java
    └── dto/
        ├── CreateOrderRequest.java
        └── OrderResponse.java
```

Suffix conventions

<Noun>

Domain object — Order, Invoice, Customer

<Noun>Validator

Validator — OrderValidator

<Noun>Service

Service — OrderService

<Noun>Repository

Repository — OrderRepository

<Noun>Request / Response

DTO — CreateOrderRequest

Cheat sheet: one line per stereotype



Domain Object

Represents a business concept and enforces its own rules



Validator

Answers "is this input acceptable?" — no side effects



Service

Orchestrates a use case and owns the transaction



Repository / DAO

Reads and writes storage — nothing else

Questions?

Before you write your next class, ask: which stereotype is this?

Domain Objects · Validators · Services · Repositories